

MODUL 13

PENGUNAAN *SENSOR* DAN *DEPLOY* APLIKASI FLUTTER

A. Capaian Pembelajaran

Mahasiswa mampu memahami dan membuat aplikasi menggunakan *sensor camera* dan GPS pada *smartphone* Android kemudian dapat menerapkan (*deploying*) aplikasi Flutter untuk dipublikasikan (*publishing*) ke *App Store*.

B. Tujuan Pembelajaran

Pada modul 13, akan membahas secara detail hal-hal yang diperlukan dalam rangka membuat aplikasi *smartphone* berbasis Android dengan *framework* Flutter, dengan tujuan:

1. Dapat menggunakan *sensor camera* pada *smartphone* Android.
2. Dapat menggunakan *sensor* GPS pada *smartphone* Android.
3. Dapat menerapkan dan mempublikasikan ke *App Store*.

C. Teori & Praktek

Sebagian besar perangkat Android memiliki *sensor built-in* yang mengukur gerakan, orientasi, dan berbagai kondisi lingkungan. Sensor ini mampu menyediakan data mentah dengan presisi dan akurasi tinggi, dan berguna jika Anda ingin memantau *positioning* atau pergerakan tiga dimensi perangkat, atau memantau perubahan di lingkungan sekitar di dekat perangkat. Misalnya, suatu *game* mungkin akan melacak pembacaan dari *sensor* gravitasi perangkat untuk menyimpulkan gerakan dan *gestur* pengguna yang kompleks, seperti kemiringan, goyangan, rotasi, atau ayunan. Demikian juga, aplikasi cuaca mungkin menggunakan *sensor* suhu dan *sensor* kelembapan perangkat untuk menghitung dan melaporkan titik embun, atau suatu aplikasi perjalanan mungkin menggunakan *sensor* medan *geomagnetik* dan akselerometer untuk melaporkan arah kompas.

Platform Android mendukung tiga kategori sensor yang luas:

- *Sensor* gerak
Sensor ini mengukur gaya akselerasi dan gaya rotasi pada tiga sumbu. Kategori ini mencakup *akselerometer*, *sensor* gravitasi, *giroskop*, dan *sensor* vektor rotasi.
- *Sensor* lingkungan

Sensor ini mengukur berbagai parameter lingkungan, seperti tekanan dan suhu udara sekitar, pencahayaan, serta kelembapan. Kategori ini meliputi *barometer*, *fotometer*, dan *termometer*.

- *Sensor* posisi

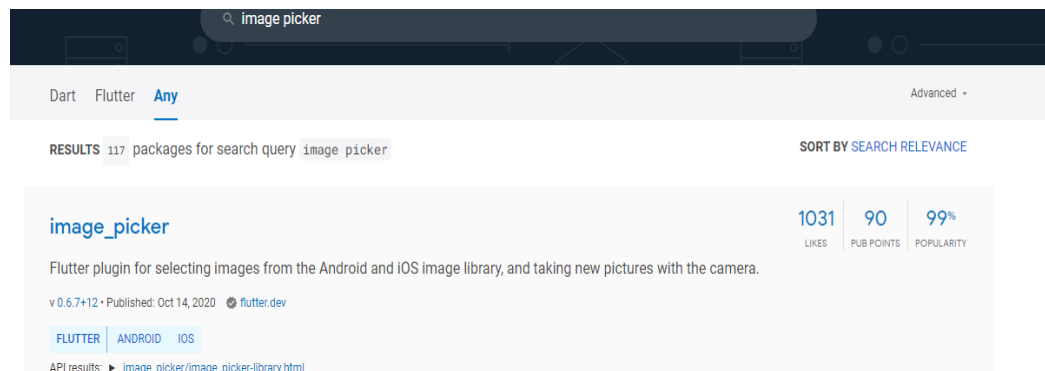
Sensor ini mengukur posisi fisik perangkat. Kategori ini meliputi sensor orientasi dan *magnetometer*.

Sensor GPS digunakan untuk menentukan posisi pengguna dari hasil kalkulasi *trigonometri* berdasarkan sinyal-sinyal dari satelit terdekat yang berhasil diterima *smartphone*. Dengan mengaktifkannya, *smartphone* bisa melacak keberadaan pengguna ataupun orang lain serta digunakan sebagai sistem navigasi yang akan memandu pengguna untuk melihat atau menunjukkan suatu lokasi.

Sensor kamera adalah *sensor* penangkap gambar yang dikenal juga sebagai CCD (Charged Coupled Device) dan CMOS (Complementary Metal Oxide Semiconductor) dan yang terbaru BSI-CMOS (Back Side Illumination CMOS) yang terdiri dari jutaan *pixel* (MP-mega pixel) lebih.

Latihan 1 (Menggunakan Camera)

1. Buat *Project* baru pada Android Studio ataupun Visual Studio Code beri nama : “flutter_camera”.
2. Buka situs **pub.dev** untuk mencari *package camera* dengan nama **image_picker**.



3. Tambahkan *package* tersebut ke dalam **pubspec.yaml** dan ketik *flutter pub get* pada *terminal*.

4. Pada `_MyHomePageState` buat *variable* untuk penggunaan *image view*-nya.

```
PickedFile _imageFile;
dynamic _pickImageError;

final ImagePicker _picker = ImagePicker();
String _retrieveDataError;

final TextEditingController maxWidthController = TextEditingController();
final TextEditingController maxHeightController = TextEditingController();
final TextEditingController qualityController = TextEditingController();
```

5. Buat fungsi untuk mengatasi penerimaan *lostdata*.

```
Future<void> retrieveLostData() async {
  final LostData response = await _picker.getLostData();
  if (response.isEmpty) {
    return;
  }
  if (response.file != null) {
    setState(() {
      _imageFile = response.file;
    });
  } else {
    _retrieveDataError = response.exception.code;
  }
}
```

6. Buat fungsi untuk mengatasi *errorWidget* pada saat *rendering*.

```
Text _getRetrieveErrorWidget() {
  if (_retrieveDataError != null) {
    final Text result = Text(_retrieveDataError);
    _retrieveDataError = null;
    return result;
  }
  return null;
}
```

7. Buat fungsi untuk pengambilan gambar seperti dibawah ini.

```
void _onImageButtonPressed(ImageSource source, {BuildContext context})
async {
  await _displayPickImageDialog(context,
    (double maxWidth, double maxHeight, int quality) async {
      try {
        final pickedFile = await _picker.getImage(
          source: source,
          maxWidth: maxWidth,
          maxHeight: maxHeight,
```

```

        imageQuality: quality,
    );
    setState(() {
      _imageFile = pickedFile;
    });
  } catch (e) {
    setState(() {
      _pickImageError = e;
    });
  }
});
}

```

8. Buat fungsi *previewImage* seperti dibawah ini.

```

Widget _previewImage() {
  final Text retrieveError = _getRetrieveErrorWidget();
  if (retrieveError != null) {
    return retrieveError;
  }
  if (_imageFile != null) {
    if (kIsWeb) {
      // Why network?
      // See https://pub.dev/packages/image_picker#getting-ready-for-the-
web-platform
      return Image.network(_imageFile.path);
    } else {
      return Image.file(File(_imageFile.path));
    }
  } else if (_pickImageError != null) {
    return Text(
      'Pick image error: $_pickImageError',
      textAlign: TextAlign.center,
    );
  } else {
    return const Text(
      'You have not yet picked an image.',
      textAlign: TextAlign.center,
    );
  }
}

```

9. Buat fungsi *dialog* untuk mengatur tinggi, lebar, dan kualitas gambar sebelum di buat.

```

Future<void> _displayPickImageDialog(
  BuildContext context, OnPickImageCallback onPick) async {
  return showDialog(
    context: context,
    builder: (context) {
      return AlertDialog(
        title: Text('Add optional parameters'),
        content: Column(
          children: <Widget>[
            TextField(

```

```

        controller: maxWidthController,
        keyboardType: TextInputType.numberWithOptions(decimal:
true),
        decoration:
        InputDecoration(hintText: "Enter maxWidth if desired"),
    ),
    TextField(
        controller: maxHeightController,
        keyboardType: TextInputType.numberWithOptions(decimal:
true),
        decoration:
        InputDecoration(hintText: "Enter maxHeight if desired"),
    ),
    TextField(
        controller: qualityController,
        keyboardType: TextInputType.number,
        decoration:
        InputDecoration(hintText: "Enter quality if desired"),
    ),
  ],
),
actions: <Widget>[
  FlatButton(
    child: const Text('CANCEL'),
    onPressed: () {
      Navigator.of(context).pop();
    },
  ),
  FlatButton(
    child: const Text('PICK'),
    onPressed: () {
      double width = maxWidthController.text.isNotEmpty
        ? double.parse(maxWidthController.text)
        : null;
      double height = maxHeightController.text.isNotEmpty
        ? double.parse(maxHeightController.text)
        : null;
      int quality = qualityController.text.isNotEmpty
        ? int.parse(qualityController.text)
        : null;
      onPick(width, height, quality);
      Navigator.of(context).pop();
    },
  ),
],
);
});
}
}
typedef void OnPickImageCallback(
  double maxWidth, double maxHeight, int quality);

```

10. Modifikasi pada *widget build*. Seperti dibawah ini.

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text(widget.title),
    ),
    body: Center(
      child: !kIsWeb && defaultTargetPlatform == TargetPlatform.android
        ? FutureBuilder<void>(
          future: retrieveLostData(),
          builder: (BuildContext context, AsyncSnapshot<void> snapshot) {
            switch (snapshot.connectionState) {
              case ConnectionState.none:
              case ConnectionState.waiting:
                return const Text(
                  'You have not yet picked an image.',
                  textAlign: TextAlign.center,
                );
              case ConnectionState.done:
                return _previewImage();
              default:
                if (snapshot.hasError) {
                  return Text(
                    'Pick image/video error: ${snapshot.error}',
                    textAlign: TextAlign.center,
                  );
                } else {
                  return const Text(
                    'You have not yet picked an image.',
                    textAlign: TextAlign.center,
                  );
                }
            }
          },
        )
        : (_previewImage()),
    ),
    floatingActionButton: Column(
      mainAxisAlignment: MainAxisAlignment.end,
      children: <Widget>[
        FloatingActionButton(
          onPressed: () {
            _onImageButtonPressed(ImageSource.gallery, context: context);
          },
          heroTag: 'image0',
          tooltip: 'Pick Image from gallery',
          child: const Icon(Icons.photo_library),
        ),
        Padding(
          padding: const EdgeInsets.only(top: 16.0),
          child: FloatingActionButton(
            onPressed: () {
              _onImageButtonPressed(ImageSource.camera, context: context);
            },
          ),
        ),
      ],
    ),
  );
}

```

```

    },
    heroTag: 'image1',
    tooltip: 'Take a Photo',
    child: const Icon(Icons.camera_alt),
  ),
),
Padding(
  padding: const EdgeInsets.only(top: 16.0),
  child: FloatingActionButton(
    backgroundColor: Colors.red,
    onPressed: () {

      _onImageButtonPressed(ImageSource.gallery);
    },
    heroTag: 'video0',
    tooltip: 'Pick Video from gallery',
    child: const Icon(Icons.video_library),
  ),
),
],
),
);
}

```

11. Kode lengkapnya bisa dilihat seperti dibawah ini.

```

// Copyright 2019 The Flutter Authors. All rights reserved.
// Use of this source code is governed by a BSD-style license that can be
// found in the LICENSE file.

// ignore_for_file: public_member_api_docs

import 'dart:async';
import 'dart:io';

import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter/src/widgets/basic.dart';
import 'package:flutter/src/widgets/container.dart';
import 'package:image_picker/image_picker.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Image Picker Demo',
      home: MyHomePage(title: 'Image Picker Example'),
    );
  }
}

```

```

}

class MyHomePage extends StatefulWidget {
  MyHomePage({Key key, this.title}) : super(key: key);

  final String title;

  @override
  _MyHomePageState createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  PickedFile _imageFile;
  dynamic _pickImageError;

  final ImagePicker _picker = ImagePicker();
  String _retrieveDataError;

  final TextEditingController maxWidthController = TextEditingController();
  final TextEditingController maxHeightController =
TextEditingController();
  final TextEditingController qualityController = TextEditingController();

  Future<void> retrieveLostData() async {
    final LostData response = await _picker.getLostData();
    if (response.isEmpty) {
      return;
    }
    if (response.file != null) {

      setState(() {
        _imageFile = response.file;
      });

    } else {
      _retrieveDataError = response.exception.code;
    }
  }

  void _onImageButtonPressed(ImageSource source, {BuildContext context})
  async {
    await _displayPickImageDialog(context,
      (double maxWidth, double maxHeight, int quality) async {
        try {
          final pickedFile = await _picker.getImage(
            source: source,
            maxWidth: maxWidth,
            maxHeight: maxHeight,
            imageQuality: quality,
          );
          setState(() {
            _imageFile = pickedFile;
          });
        }
      }
    );
  }
}

```



```

        } catch (e) {
          setState(() {
            _pickImageError = e;
          });
        }
      });
    }

    @override
    void dispose() {

      maxWidthController.dispose();
      maxHeightController.dispose();
      qualityController.dispose();
      super.dispose();
    }

    Text _getRetrieveErrorWidget() {
      if (_retrieveDataError != null) {
        final Text result = Text(_retrieveDataError);
        _retrieveDataError = null;
        return result;
      }
      return null;
    }

    Widget _previewImage() {
      final Text retrieveError = _getRetrieveErrorWidget();
      if (retrieveError != null) {
        return retrieveError;
      }
      if (_imageFile != null) {
        if (kIsWeb) {
          // Why network?
          // See https://pub.dev/packages/image\_picker#getting-ready-for-the-web-platform
          return Image.network(_imageFile.path);
        } else {
          return Image.file(File(_imageFile.path));
        }
      } else if (_pickImageError != null) {
        return Text(
          'Pick image error: $_pickImageError',
          textAlign: TextAlign.center,
        );
      } else {
        return const Text(
          'You have not yet picked an image.',
          textAlign: TextAlign.center,
        );
      }
    }
  }
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text(widget.title),
    ),
    body: Center(
      child: !kIsWeb && defaultTargetPlatform == TargetPlatform.android
        ? FutureBuilder<void>(
            future: retrieveLostData(),
            builder: (BuildContext context, AsyncSnapshot<void> snapshot) {
              switch (snapshot.connectionState) {
                case ConnectionState.none:
                case ConnectionState.waiting:
                  return const Text(
                    'You have not yet picked an image.',
                    textAlign: TextAlign.center,
                  );
                case ConnectionState.done:
                  return _previewImage();
                default:
                  if (snapshot.hasError) {
                    return Text(
                      'Pick image/video error: ${snapshot.error}',
                      textAlign: TextAlign.center,
                    );
                  } else {
                    return const Text(
                      'You have not yet picked an image.',
                      textAlign: TextAlign.center,
                    );
                  }
              }
            },
          )
        : (_previewImage()),
    ),
    floatingActionButton: Column(
      mainAxisAlignment: MainAxisAlignment.end,
      children: <Widget>[
        FloatingActionButton(
          onPressed: () {
            _onImageButtonPressed(ImageSource.gallery, context: context);
          },
          heroTag: 'image0',
          tooltip: 'Pick Image from gallery',
          child: const Icon(Icons.photo_library),
        ),
        Padding(
          padding: const EdgeInsets.only(top: 16.0),
          child: FloatingActionButton(
            onPressed: () {
              _onImageButtonPressed(ImageSource.camera, context:

```

```

context);
    },
    heroTag: 'image1',
    tooltip: 'Take a Photo',
    child: const Icon(Icons.camera_alt),
  ),
),
Padding(
  padding: const EdgeInsets.only(top: 16.0),
  child: FloatingActionButton(
    backgroundColor: Colors.red,
    onPressed: () {
      _onImageButtonPressed(ImageSource.gallery);
    },
    heroTag: 'video0',
    tooltip: 'Pick Video from gallery',
    child: const Icon(Icons.video_library),
  ),
),
],
),
);
}

Future<void> _displayPickImageDialog(
  BuildContext context, OnPickImageCallback onPick) async {
  return showDialog(
    context: context,
    builder: (context) {
      return AlertDialog(
        title: Text('Add optional parameters'),
        content: Column(
          children: <Widget>[
            TextField(
              controller: maxWidthController,
              keyboardType: TextInputType.numberWithOptions(decimal:
true),
              decoration:
                InputDecoration(hintText: "Enter maxWidth if desired"),
            ),
            TextField(
              controller: maxHeightController,
              keyboardType: TextInputType.numberWithOptions(decimal:
true),
              decoration:
                InputDecoration(hintText: "Enter maxHeight if desired"),
            ),
            TextField(
              controller: qualityController,
              keyboardType: TextInputType.number,
              decoration:
                InputDecoration(hintText: "Enter quality if desired"),
            ),
          ],
        ),
      );
    },
  );
}

```

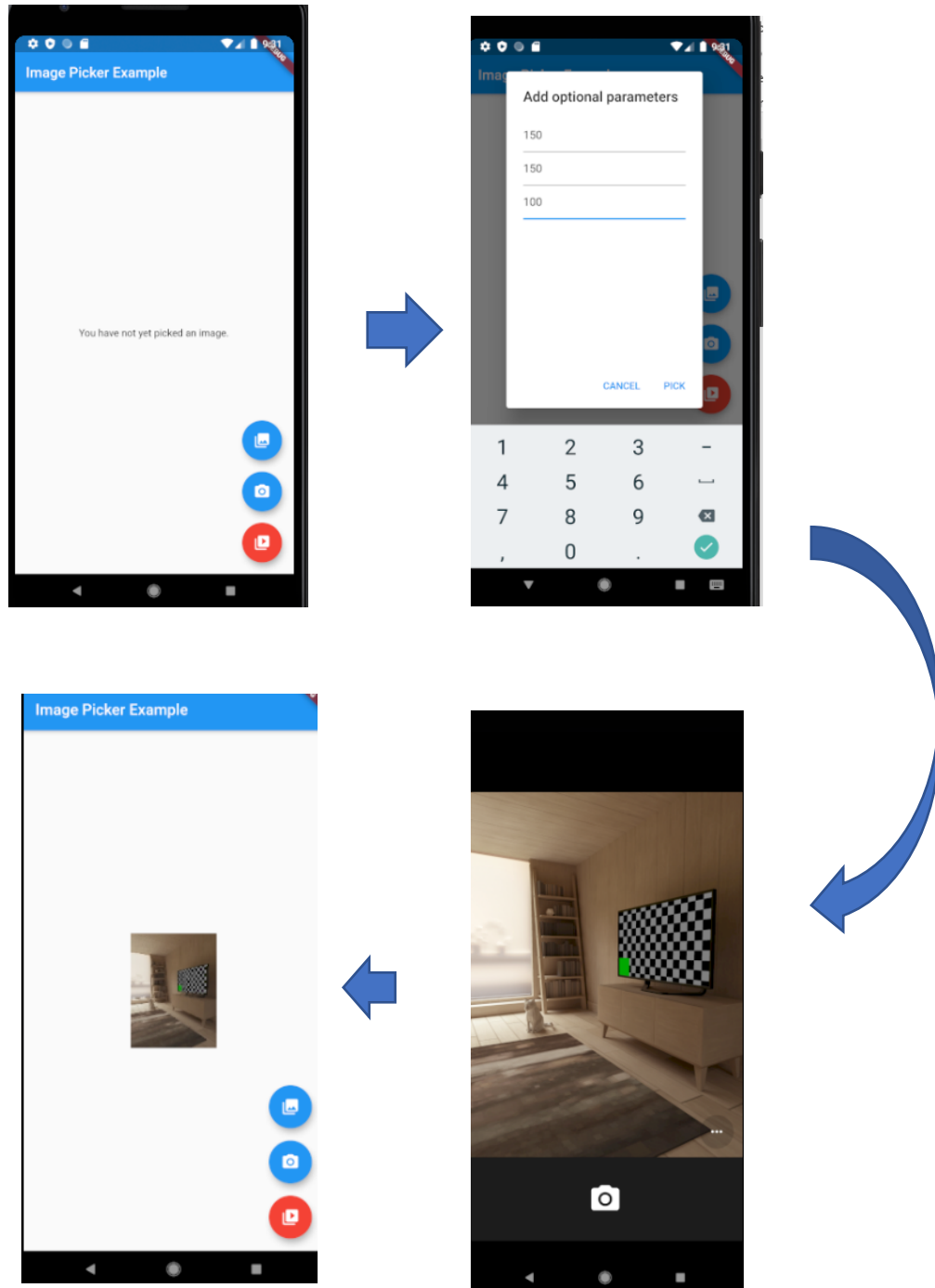
```

    ],
  ),
  actions: <Widget>[
    FlatButton(
      child: const Text('CANCEL'),
      onPressed: () {
        Navigator.of(context).pop();
      },
    ),
    FlatButton(
      child: const Text('PICK'),
      onPressed: () {
        double width = maxWidthController.text.isNotEmpty
          ? double.parse(maxWidthController.text)
          : null;
        double height = maxHeightController.text.isNotEmpty
          ? double.parse(maxHeightController.text)
          : null;
        int quality = qualityController.text.isNotEmpty
          ? int.parse(qualityController.text)
          : null;
        onPick(width, height, quality);
        Navigator.of(context).pop();
      },
    ),
  ],
);
});
}
}

typedef void OnPickImageCallback(
  double maxWidth, double maxHeight, int quality);

```

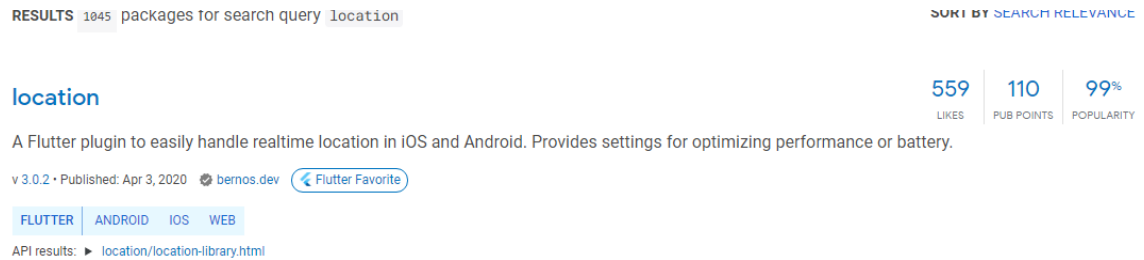
12. Jalankan kode program tersebut hingga *output* seperti dibawah ini.



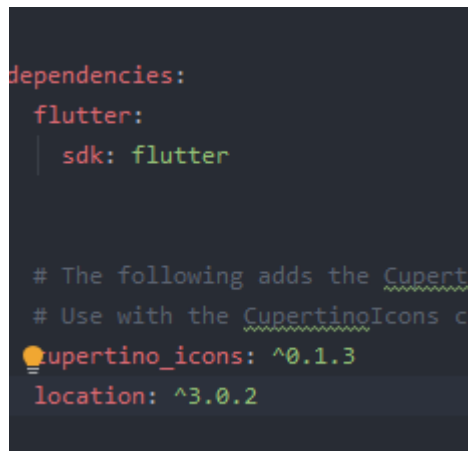
Latihan 2 (Get GPS location)

1. Buat *Project* baru pada Android Studio ataupun Visual Studio Code beri nama :
"flutter_location".

2. Buka situs **pub.dev** untuk mencari *package camera* dengan nama *location*.



3. Tambahkan *package* tersebut kedalam **pubspec.yaml** dan ketik *flutter pub get* pada *terminal*.



4. Pada *_MyHomePageState* buat *variable* untuk penggunaan *location*-nya.

```
final Location location = Location();
```

5. Pada *widget build* modifkasi seperti gambar dibawah ini.

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text(widget.title),
    ),
    body: Container(
      padding: const EdgeInsets.all(32),
      child: Column(
        children: const <Widget>[
          PermissionStatusWidget(),
          Divider(height: 32),
          ServiceEnabledWidget(),
          Divider(height: 32),
          GetLocationWidget(),
          Divider(height: 32),
          ListenLocationWidget()
        ],
      ),
    ),
  );
}
```

```

    ),
  );
}
}

```

6. Buat class *statefullWidget* untuk *PermissionStatus*. Seperti gambar dibawah ini.

```

class PermissionStatusWidget extends StatefulWidget {
  const PermissionStatusWidget({Key key}) : super(key: key);

  @override
  _PermissionStatusState createState() => _PermissionStatusState();
}

class _PermissionStatusState extends State<PermissionStatusWidget> {
  final Location location = Location();

  PermissionStatus _permissionGranted;

  Future<void> _checkPermissions() async {
    final PermissionStatus permissionGrantedResult =
    await location.hasPermission();
    setState(() {
      _permissionGranted = permissionGrantedResult;
    });
  }

  Future<void> _requestPermission() async {
    if (_permissionGranted != PermissionStatus.granted) {
      final PermissionStatus permissionRequestedResult =
      await location.requestPermission();
      setState(() {
        _permissionGranted = permissionRequestedResult;
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: <Widget>[
        Text(
          'Permission status: ${_permissionGranted ?? "unknown"}',
          style: Theme.of(context).textTheme.body2,
        ),
        Row(
          children: <Widget>[
            Container(
              margin: const EdgeInsets.only(right: 42),
              child: RaisedButton(
                child: const Text('Check'),
                onPressed: _checkPermissions,
              ),

```

```

    ),
    RaisedButton(
      child: const Text('Request'),
      onPressed: _permissionGranted == PermissionStatus.granted
        ? null
        : _requestPermission,
    ),
  ],
),
],
);
}
}

```

7. Buat class *StateFullWidget ServiceEnable* seperti dibawah ini.

```

class ServiceEnabledWidget extends StatefulWidget {
  const ServiceEnabledWidget({Key key}) : super(key: key);

  @override
  _ServiceEnabledState createState() => _ServiceEnabledState();
}

class _ServiceEnabledState extends State<ServiceEnabledWidget> {
  final Location location = Location();

  bool _serviceEnabled;

  Future<void> _checkService() async {
    final bool serviceEnabledResult = await location.serviceEnabled();
    setState(() {
      _serviceEnabled = serviceEnabledResult;
    });
  }

  Future<void> _requestService() async {
    if (_serviceEnabled == null || !_serviceEnabled) {
      final bool serviceRequestedResult = await location.requestService();
      setState(() {
        _serviceEnabled = serviceRequestedResult;
      });
      if (!serviceRequestedResult) {
        return;
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: <Widget>[
        Text('Service enabled: ${_serviceEnabled ?? "unknown"}',
          style: Theme.of(context).textTheme.body2),
      ],
    );
  }
}

```



```

    Row(
      children: <Widget>[
        Container(
          margin: const EdgeInsets.only(right: 42),
          child: RaisedButton(
            child: const Text('Check'),
            onPressed: _checkService,
          ),
        ),
        RaisedButton(
          child: const Text('Request'),
          onPressed: _serviceEnabled == true ? null : _requestService,
        ),
      ],
    ),
  ],
);
}
}

```

8. Buat class *StatefulWidget getLocation* seperti di bawah ini.

```

class GetLocationWidget extends StatefulWidget {
  const GetLocationWidget({Key key}) : super(key: key);

  @override
  _GetLocationState createState() => _GetLocationState();
}

class _GetLocationState extends State<GetLocationWidget> {
  final Location location = Location();

  LocationData _location;
  String _error;

  Future<void> _getLocation() async {
    setState(() {
      _error = null;
    });
    try {
      final LocationData _locationResult = await location.getLocation();
      setState(() {
        _location = _locationResult;
      });
    } on PlatformException catch (err) {
      setState(() {
        _error = err.code;
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return Column(

```

```

crossAxisAlignment: CrossAxisAlignment.start,
children: <Widget>[
  Text(
    'Location: ' + (_error ?? '${_location ?? "unknown"}'),
    style: Theme.of(context).textTheme.body2,
  ),
  Row(
    children: <Widget>[
      RaisedButton(
        child: const Text('Get'),
        onPressed: _getLocation,
      ),
    ],
  ),
],
);
}
}

```

9. Buat class *StatefullWidget* listenLcoation seperti di bawah ini.

```

class ListenLocationWidget extends StatefulWidget {
  const ListenLocationWidget({Key key}) : super(key: key);

  @override
  _ListenLocationState createState() => _ListenLocationState();
}

class _ListenLocationState extends State<ListenLocationWidget> {
  final Location location = Location();

  LocationData _location;
  StreamSubscription<LocationData> _locationSubscription;
  String _error;

  Future<void> _listenLocation() async {
    _locationSubscription =
      location.onLocationChanged.handleError((dynamic err) {
        setState(() {
          _error = err.code;
        });
        _locationSubscription.cancel();
      }).listen((LocationData currentLocation) {
        setState(() {
          _error = null;

          _location = currentLocation;
        });
      });
  }

  Future<void> _stopListen() async {
    _locationSubscription.cancel();
  }
}

```

```

@override
Widget build(BuildContext context) {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: <Widget>[
      Text(
        'Listen location: ' + (_error ?? '${_location} ?? "unknown"'),
        style: Theme.of(context).textTheme.body2,
      ),
      Row(
        children: <Widget>[
          Container(
            margin: const EdgeInsets.only(right: 42),
            child: RaisedButton(
              child: const Text('Listen'),
              onPressed: _listenLocation,
            ),
          ),
          RaisedButton(
            child: const Text('Stop'),
            onPressed: _stopListen,
          ),
        ],
      ),
    ],
  );
}

```

10. Untuk *full code* bisa dilihat dibawah ini.

```

import 'dart:async';

import 'package:flutter/material.dart';
import 'package:location/location.dart';
import 'package:flutter/services.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Location',
      theme: ThemeData(
        primarySwatch: Colors.amber,
      ),
      home: const MyHomePage(title: 'Flutter Location Demo'),
    );
  }
}

```

```

class MyHomePage extends StatefulWidget {
  const MyHomePage({Key key, this.title}) : super(key: key);
  final String title;

  @override
  _MyHomePageState createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  final Location location = Location();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.title),
      ),
      body: Container(
        padding: const EdgeInsets.all(32),
        child: Column(
          children: const <Widget>[
            PermissionStatusWidget(),
            Divider(height: 32),
            ServiceEnabledWidget(),
            Divider(height: 32),
            GetLocationWidget(),
            Divider(height: 32),
            ListenLocationWidget()
          ],
        ),
      ),
    );
  }
}

class PermissionStatusWidget extends StatefulWidget {
  const PermissionStatusWidget({Key key}) : super(key: key);

  @override
  _PermissionStatusState createState() => _PermissionStatusState();
}

class _PermissionStatusState extends State<PermissionStatusWidget> {
  final Location location = Location();

  PermissionStatus _permissionGranted;

  Future<void> _checkPermissions() async {
    final PermissionStatus permissionGrantedResult =
      await location.hasPermission();
    setState(() {
      _permissionGranted = permissionGrantedResult;
    });
  }
}

```

```

    }

    Future<void> _requestPermission() async {
      if (_permissionGranted != PermissionStatus.granted) {
        final PermissionStatus permissionRequestedResult =
          await location.requestPermission();
        setState(() {
          _permissionGranted = permissionRequestedResult;
        });
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: <Widget>[
        Text(
          'Permission status: ${_permissionGranted ?? "unknown"}',
          style: Theme.of(context).textTheme.body2,
        ),
        Row(
          children: <Widget>[
            Container(
              margin: const EdgeInsets.only(right: 42),
              child: RaisedButton(
                child: const Text('Check'),
                onPressed: _checkPermissions,
              ),
            ),
            RaisedButton(
              child: const Text('Request'),
              onPressed: _permissionGranted == PermissionStatus.granted
                ? null
                : _requestPermission,
            ),
          ],
        ),
      ],
    );
  }
}

class ServiceEnabledWidget extends StatefulWidget {
  const ServiceEnabledWidget({Key key}) : super(key: key);

  @override
  _ServiceEnabledState createState() => _ServiceEnabledState();
}

class _ServiceEnabledState extends State<ServiceEnabledWidget> {
  final Location location = Location();

  bool _serviceEnabled;

```

```

Future<void> _checkService() async {
  final bool serviceEnabledResult = await location.serviceEnabled();
  setState(() {
    _serviceEnabled = serviceEnabledResult;
  });
}

Future<void> _requestService() async {
  if (_serviceEnabled == null || !_serviceEnabled) {
    final bool serviceRequestedResult = await location.requestService();
    setState(() {
      _serviceEnabled = serviceRequestedResult;
    });
    if (!serviceRequestedResult) {
      return;
    }
  }
}

@override
Widget build(BuildContext context) {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: <Widget>[
      Text('Service enabled: ${_serviceEnabled ?? "unknown"}',
        style: Theme.of(context).textTheme.body2),
      Row(
        children: <Widget>[
          Container(
            margin: const EdgeInsets.only(right: 42),
            child: RaisedButton(
              child: const Text('Check'),
              onPressed: _checkService,
            ),
          ),
          RaisedButton(
            child: const Text('Request'),
            onPressed: _serviceEnabled == true ? null : _requestService,
          ),
        ],
      ),
    ],
  );
}

class GetLocationWidget extends StatefulWidget {
  const GetLocationWidget({Key key}) : super(key: key);

  @override
  _GetLocationState createState() => _GetLocationState();
}

class _GetLocationState extends State<GetLocationWidget> {
  final Location location = Location();

```

```

    LocationData _location;
    String _error;

    Future<void> _getLocation() async {
      setState(() {
        _error = null;
      });
      try {
        final LocationData _locationResult = await location.getLocation();
        setState(() {
          _location = _locationResult;
        });
      } on PlatformException catch (err) {
        setState(() {
          _error = err.code;
        });
      }
    }

    @override
    Widget build(BuildContext context) {
      return Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          Text(
            'Location: ' + (_error ?? '${_location ?? "unknown"}'),
            style: Theme.of(context).textTheme.body2,
          ),
          Row(
            children: <Widget>[
              RaisedButton(
                child: const Text('Get'),
                onPressed: _getLocation,
              ),
            ],
          ),
        ],
      );
    }
  }

  class ListenLocationWidget extends StatefulWidget {
    const ListenLocationWidget({Key key}) : super(key: key);

    @override
    _ListenLocationState createState() => _ListenLocationState();
  }

  class _ListenLocationState extends State<ListenLocationWidget> {
    final Location location = Location();

    LocationData _location;
    StreamSubscription<LocationData> _locationSubscription;
    String _error;

```

```

Future<void> _listenLocation() async {
  _locationSubscription =
    location.onLocationChanged.handleError((dynamic err) {
      setState(() {
        _error = err.code;
      });
      _locationSubscription.cancel();
    }).listen((LocationData currentLocation) {
      setState(() {
        _error = null;

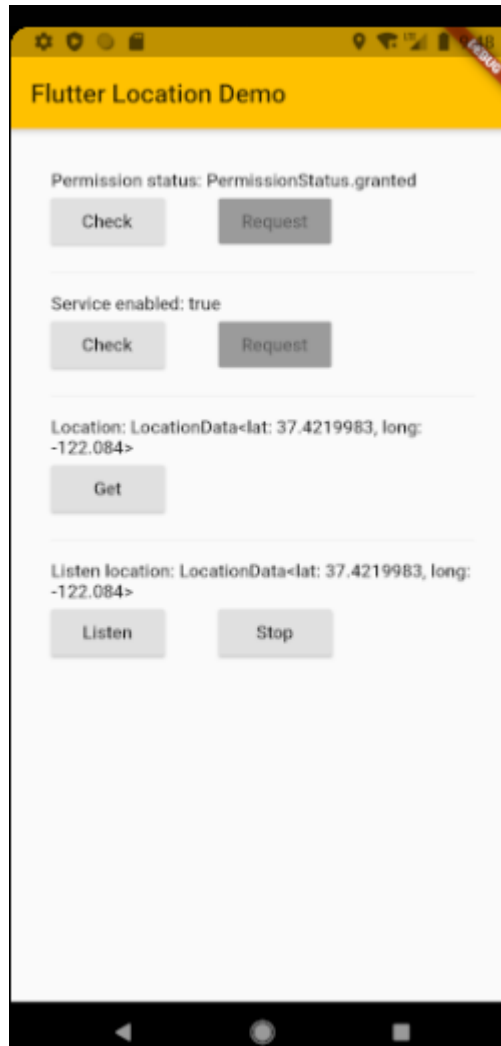
        _location = currentLocation;
      });
    });
}

Future<void> _stopListen() async {
  _locationSubscription.cancel();
}

@override
Widget build(BuildContext context) {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: <Widget>[
      Text(
        'Listen location: ' + (_error ?? '${_location} ?? "unknown"'),
        style: Theme.of(context).textTheme.body2,
      ),
      Row(
        children: <Widget>[
          Container(
            margin: const EdgeInsets.only(right: 42),
            child: RaisedButton(
              child: const Text('Listen'),
              onPressed: _listenLocation,
            ),
          ),
          RaisedButton(
            child: const Text('Stop'),
            onPressed: _stopListen,
          ),
        ],
      ),
    ],
  );
}

```

11. Jalankan program tersebut hingga hasilnya seperti ini.



Latihan 3 (Deploy Aplikasi Flutter di Android atau Release APK di Flutter)

1. Android Manifest

- Buka **android/app/src/AndroidManifest.xml**
- Ubah nama **package**, **android:label**

2. Buat Keystore

Di dalam *cmd folder project* tulis perintah :

```
keytool -genkey -v -keystore keystore.jks -keyalg RSA -keysize 2048 -validity 10000 -alias key
```

3. Reference Keystore

Buat *file* **android/key.properties** :

```
storePassword=<password from previous step>
```

```
keyPassword=<password from previous step>
keyAlias=key
storeFile=<location of the key store file, e.g. /Users/<user
name>/key.jks>
```

4. Ubah Gradle

- Buka **android/app/build.gradle**
- Ubah **defaultConfig**:

```
defaultConfig {
    applicationId "id.athalon.hello" //nama package
    minSdkVersion 21
    targetSdkVersion 30
    versionCode 1 //version code
    versionName "1.0" //version code
    testInstrumentationRunner "android.support.test.runner.AndroidJ
UnitRunner" }
Ubah bagian ini :
android {

Menjadi :
def keystoreProperties = new Properties()
def keystorePropertiesFile = rootProject.file('key.properties')
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(new FileInputStream(keystorePropertiesFile))
}
android {

Ubah bagian ini :
buildTypes {
    release {
        signingConfig signingConfigs.debug
    }
}

Menjadi :
signingConfigs {
    release {
        keyAlias keystoreProperties['keyAlias']
        keyPassword keystoreProperties['keyPassword']
        storeFile file(keystoreProperties['storeFile'])
        storePassword keystoreProperties['storePassword']
    }
}

buildTypes {
    release {
        signingConfig signingConfigs.release
    }
}
```

5. Jalankan di cmd project

- flutter clean
- flutter build apk --release

Latihan 4 (Deploy Aplikasi Flutter di iOS)

Seperti dalam kasus Google *Play Store*, Apple juga mengikuti pedoman *publishing* aplikasinya sendiri. Harap pastikan untuk membaca semua informasi mengenai hal yang sama, sebelum membangun aplikasi. Berikut adalah tautan yang dapat Anda periksa untuk membaca detail lebih lanjut tentang *publishing* aplikasi Apple: <https://developer.apple.com/app-store/review/>. Setelah aplikasi dikirimkan, seperti dalam kasus Google juga, Apple akan memeriksa aplikasi untuk mematuhi pedoman publishing. Perhatikan bahwa Flutter mendukung iOS 8.0 dan yang lebih baru. Ini penting untuk diketahui saat set Xcode untuk pembuatan *build*.

Apple akan menggunakan ***App Store Connect***, yang sebelumnya dikenal sebagai *iTunes. Console* ini digunakan untuk mengelola siklus hidup aplikasi. *Console* ini akan membantu Anda menyetel nama aplikasi, deskripsi, dan *screenshots* aplikasi, yang akan dipublikasikan bersama dengan aplikasi, harga, dan mengelola rilis.

1. **Mendaftarkan *Bundle ID***, langkah-langkahnya adalah sebagai berikut:
 - a. Buka halaman ***App ID*** dari akun pengembang Apple Anda.
 - b. Klik *icon* + untuk membuat *Bundle ID* baru.
 - c. Ketikkan nama aplikasi dan pilih ***Explicit App ID***, lalu masukkan ID.
 - d. Pilih layanan yang akan digunakan aplikasi Anda, lalu klik ***Continue***.
 - e. Langkah selanjutnya mengkonfirmasi detailnya. Sekarang, klik ***Register*** untuk mendaftarkan *Bundle ID* Anda.
2. **Generating record aplikasi di App Store Connect**, langkahnya sebagai berikut:
 - a. Buka Apple ***App Store connect*** di browser Anda dan klik ***My Apps***.
 - b. Klik *icon* + di sudut kiri atas halaman ***My Apps page*** | ***New App***.
 - c. Di dalam layar *pop-up*, isi detail aplikasi Anda. Di bagian ***Platform***, pastikan iOS dicentang. Pada titik ini, perlu disebutkan bahwa Flutter belum mendukung tvOS. Jadi, biarkan kotak centang itu tidak dicentang. Nama aplikasi tidak boleh lebih dari 30 karakter. Di bagian SKU, tambahkan ID unik untuk aplikasi Anda yang tidak terlihat di App Store:

New App

Platforms ?
☐ iOS ☐ tvOS

Name ?

Primary Language ?
 Choose ▾

Bundle ID ?
 Choose ▾
 Register a new bundle ID on the [Developer Portal](#).

SKU ?

User Access ?
☐ Limited Access ☒ Full Access

Cancel Create

- d. Check Buat.
- e. Navigasikan ke detail aplikasi untuk aplikasi Anda yang telah dibuat menggunakan langkah sebelumnya, dan pilih **App Information** dari *sidebar*.
- f. Pilih Bundle ID di bagian **General Information**.

3. Verifikasi Pengaturan Xcode

Memverifikasi pengaturan *build-publishing* di Xcode agak sederhana dibandingkan dengan di Android Studio. Pertama, navigasikan ke pengaturan target di Xcode dan lakukan hal berikut:

1. Buka `Runner.xcworkspace` di *folder* ios aplikasi Anda, di Xcode.
2. Pilih **Runner project** dari navigator proyek Xcode, yang menampilkan *file* pengaturan aplikasi. Pilih target **Runner** dari sidebar tampilan utama.
3. Pilih tab **General**.

Informasi yang ditampilkan harus diperiksa ulang; jadi di bagian **Identity**, lihat detail berikut:

- **Display Name** : Ini adalah nama aplikasi yang akan ditampilkan di *App Store* dan di manapun nama tersebut digunakan.
- **Bundle Identifier** : Ini adalah ID aplikasi yang Anda daftarkan di *App Store Connect*, seperti yang dibahas di langkah sebelumnya.

Di bagian *Signing the app*, silakan lihat detail berikut:

- **Automatically manage signing** : Menentukan apakah Xcode harus secara otomatis mengelola *signing* dan penyediaan (*provisioning*) aplikasi. Secara *default*, ini diatur ke *True*.
- **The Team** : Pilih tim yang terkait dengan akun Pengembang Apple Anda yang terdaftar. Jika Anda ingin menambahkan lebih banyak anggota, klik **Add Account**, diikuti dengan memperbarui pengaturan.

Terakhir di bagian **Deployment**, periksa **Deployment Target** : yang menyimpan nilai untuk versi iOS minimum yang akan didukung aplikasi Anda.

4. Memilih *icon* aplikasi

Seperti dalam kasus Android Studio, bahkan dalam kasus iOS, *icon placeholder* dibuat. Jika Anda ingin memiliki *icon* sendiri, harap baca pedoman *icon* aplikasi iOS sebelum melanjutkan dengan langkah-langkah berikut:

- Pilih *Assets.xcassets* di folder *Runner*; ini akan ditampilkan di navigator proyek Xcode
- Jika *icon* Anda sudah siap, perbarui *icon placeholder* dengan *icon* aplikasi Anda sendiri yang telah dibuat
- Untuk memeriksa apakah *icon* telah diperbarui, jalankan aplikasi Anda dengan menggunakan **Flutter Run**

5. Membuat Arsip *build*

Ini adalah langkah terakhir untuk membuat arsip *build* dan kemudian mengunggahnya ke Apple Store. Di baris perintah, ikuti langkah-langkah berikut di direktori aplikasi Anda:

- Jalankan flutter build iOS untuk membuat build rilis.
- Lakukan ini hanya jika Xcode Anda di bawah versi 8.3. Untuk memastikan bahwa Xcode *refresh* konfigurasi mode rilis, *restart* Xcode Anda.

Di Xcode, gunakan langkah-langkah ini untuk mengkonfigurasi versi aplikasi dan *build*:

- a. Di Xcode, buka `Runner.xcworkspace` di folder *ios* aplikasi Anda.
- b. Pilih **Product** | **Scheme** | **Runner**.
- c. Pilih **Product** | **Destination** | **Generic iOS Device**.
- d. Pilih **Runner** di navigator proyek Xcode diikuti oleh **Runner target** di *sidebar* tampilan pengaturan.
- e. Di bagian **Identity**, perbarui versi dan juga perbarui **Build Identifier** ke nomor *build* unik. Ini digunakan untuk melacak jumlah *build* yang di-upload. Setiap *build* harus memiliki nomor *build* unik.

Langkah terakhir adalah membuat arsip **build** dan mengunggahnya ke **App Store Connect**:

- a. Pilih produk dan kemudian **Achieve** produk arsip *build*
- b. Di jendela pengatur Xcode di *sidebar* | **select the iOS app** | **select the build archive** yang baru saja Anda buat
- c. Klik tombol **Validate**
- d. Setelah arsip divalidasi, Anda dapat mengklik opsi **Upload to App store**
- e. Jika ada kesalahan apapun, buat ulang versi dan coba ulangi prosesnya lagi.

D. Daftar Pustaka

1. https://developer.android.com/guide/topics/sensors/sensors_position?hl=id
2. https://pub.dev/packages/image_picker
3. <https://pub.dev/packages?q=location>
4. Modul Google Flutter Mobile Development Quick Start Guide

E. Latihan dan Tugas

1. Modifikasi dari latihan 1, dimana foto yang diambil dari kamera dapat disimpan pada perangkat.
2. Modifikasi dari latihan 2, sehingga dapat memunculkan peta lokasinya.